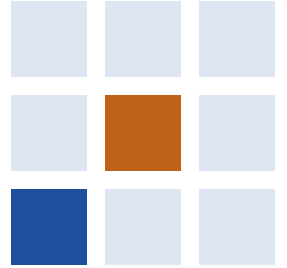


SLURM: who gets the machine

How a shared supercomputer decides who runs next, and for how long

Carlos Cotrini · Presented by Ghali



Alps, in Lugano

The largest cluster in Switzerland

Run by CSCS, the Swiss National
Supercomputing Centre

Over 1.3 million processor cores,
paired with Nvidia GH200 accelerators

Who is asking for it

The Swiss AI Initiative — ETH and EPFL

MeteoSwiss, the national weather forecast

Research groups across the Swiss
universities

It does not belong to anyone.

Hundreds of groups, all of them working on something important, all of them on a deadline.

What we will cover

1 **What is being shared**

The machine, and who is allowed near it

2 **How people ask**

Requests, queues, and pools

3 **Who goes first**

Priority, and what counts as fair

4 **Backfill**

Recovering capacity nobody was using

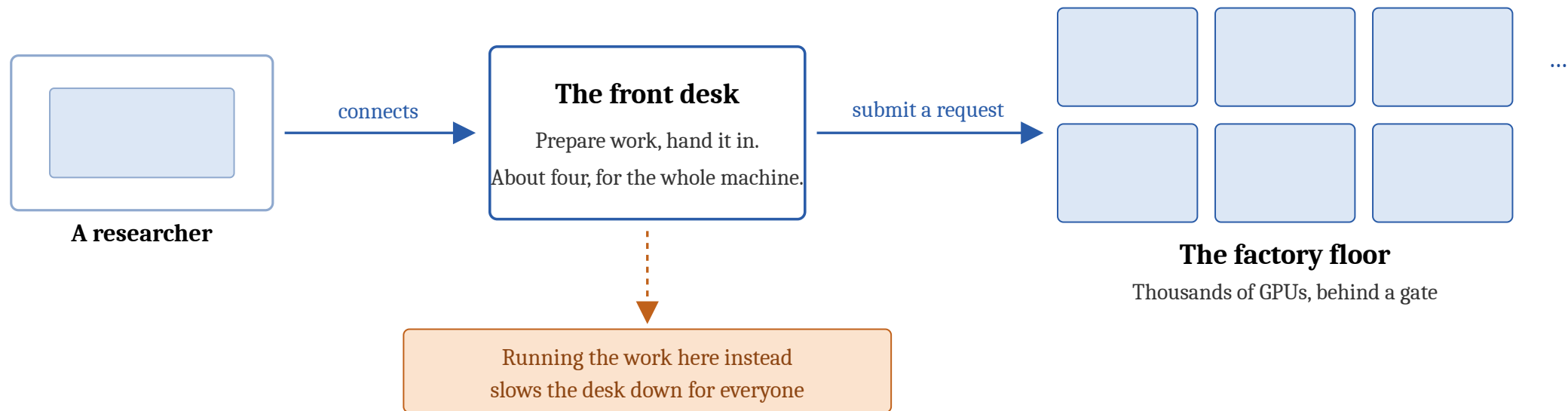


Section 1

What is being shared

1

The front desk and the factory floor



A controlled asset with a single door. **Everything through it is recorded, priced, and charged to a project.**

Thousands of computers, and they differ

Behind the gate: thousands of individual computers, called **nodes**, joined by a very fast network.

The standard node

processors + memory

GPU

GPU

GPU

GPU

Four accelerators. The expensive part, and the part everyone is waiting for.

Other nodes

processors + a great deal of memory

No accelerators. Built for different kinds of work, and priced differently.

Nobody asks which machine they can have.

They state what the work requires: processors, accelerators, memory, time. Four numbers.

What the kitchen manager does

The restaurant

The cluster	→	the restaurant
A node	→	one kitchen
Processors	→	general-purpose cooks
GPUs	→	specialised high-speed equipment
Memory	→	counter space
Storage	→	the pantry
The network	→	corridors between kitchens
SLURM	→	the kitchen manager

And what it never does

The manager does not cook.
SLURM never makes the work faster.

It answers three questions

Who gets which resources
When
And for how long

Thousands of times a day.

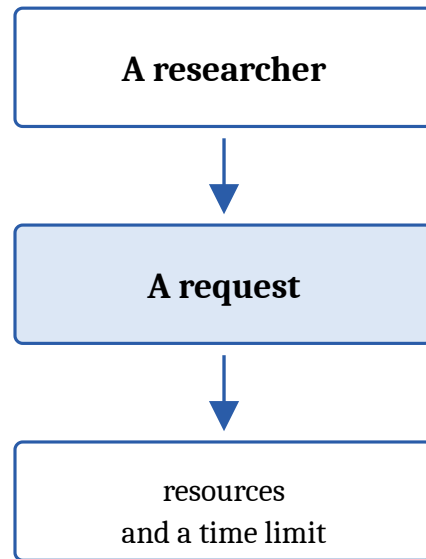


Section 2

How people ask

2

A job is a request



Named up front, with a hard limit on how long you keep them

8 processors
1 GPU
32 GB of memory
2 hours maximum

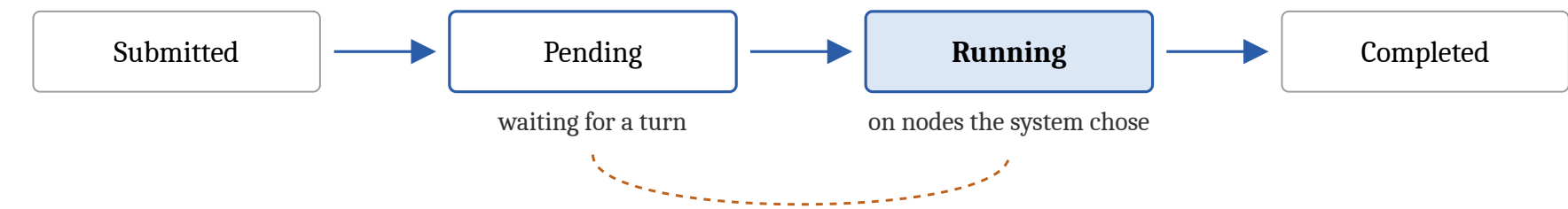
The everyday version

One oven, two cooks,
some counter space,
ninety minutes.

The work declares its needs. The system chooses the machine.

Which means capacity can be reallocated — a programme reprioritised, a project throttled, new hardware added — without anyone changing how they work.

Submitted, pending, running, completed



A queue is the healthy state.
A cluster with none was overbought.

REQUEST	ASKS FOR	STATE	WHY
A	1 GPU, 8 processors	RUNNING	it fit when it arrived
B	4 GPUs, 32 processors	PENDING	nothing that large is free
C	2 processors	PENDING	two, out of more than a million
D	1 GPU, 4 processors	PENDING	arrived last

There is certainly room for C somewhere on this machine. Under a strict queue it waits behind B, and capacity is destroyed.

Partitions: hardware plus a policy

The machine is divided into named pools, each with its own rules. Three on the cluster we are looking at.

For testing

One or two nodes
Thirty minutes maximum
debug

For the real work

Around 1200 nodes
Up to 24 hours at a time
normal · the default

For moving data

Two nodes
One per job, 24 hours
xfer

Hardware, plus a policy.

How much you may take, how long you may keep it,
and who is allowed in at all.

One of your levers.

It reads as a technical setting.
It is a business rule.

On this machine, nodes are not shared between jobs.

Ask for one node and you receive all four GPUs and all 288 cores — and are charged for all of it.

Nine lines, and what they ask for

```
#!/bin/bash
```

the shell that runs it

```
#SBATCH --account=<project>
```

which project pays

```
#SBATCH --partition=normal
```

which pool

```
#SBATCH --nodes=1
```

how many nodes

```
#SBATCH --ntasks-per-node=1
```

how many processes

```
#SBATCH --gpus-per-task=1
```

how many GPUs

```
#SBATCH --time=01:00:00
```

how long, at most

```
#SBATCH --output=logs/%x-%j.log
```

where the results land

```
srun python train.py
```

the actual science

Eight of the nine lines declare what you need and who pays. Only the last one has anything to do with the science.

Nodes are not shared



This request asks for one GPU.

It is charged for four, and for all 288 processor cores.

Three idle for an hour, paid for.



Section 3

Who goes first

3

Free nodes, and the work still waits

Two thousand requests are in the queue. There are visibly idle nodes. Both of those things are correct.



■ busy □ free

Allocation here is not first come, first served.

Every request carries a priority score, built from several factors at once.

The gap in front of you is too short.

Nodes are free, but promised to a bigger job. Yours would still be running when that promise falls due.

You are the bigger job.

More is needed than exists right now, so nodes are held for you while they accumulate — and smaller work slips past.

It is not your turn yet.

Your group has used more than its share this week, so other groups' requests sit above yours in the ordering.

Four factors, four policy questions

One score per request, recomputed every few minutes. The administrator sets the weight on each factor.

Fairshare

what this project has
used lately

entitlement enforcement

Age

how long it has waited,
up to seven days

nothing waits forever

Job size

how much is being
asked for

larger usually scores higher

Service class

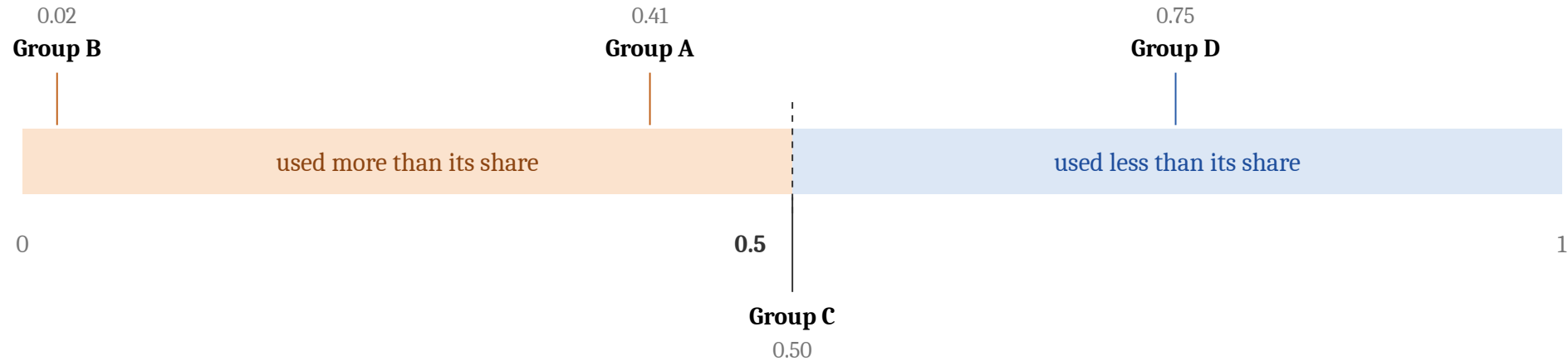
urgent, long-running,
privileged programme

granted to your account

One factor is deliberately left out: how much time you asked for.

Requesting one hour instead of twenty-four earns no priority at all. That becomes important in ten minutes.

Fairshare: 0.5 is exactly your share



It corrects itself.

Heavy users yield to light users with nobody making a decision.

Charged to the account.

A colleague's large run slows yours.
Decays with a seven-day half-life.

Age, size, and service class

The other three factors, and the question each one answers.

Age

A request climbs slowly the longer it is overtaken, and stops climbing after about seven days.

Job size

Larger requests usually score higher, because they need many nodes free at the same moment.

Service class

Named tiers an administrator defines and attaches to an account. You order from the menu you are given.

Why it exists

A group at the back of the fairshare ordering could otherwise be starved indefinitely.

Which way round is a choice

A site optimising for throughput can invert it and favour short, small work instead.

A plausible menu

urgent 10× priority, capped at 2 h and 4 nodes

long half priority, up to 7 days

operational a guaranteed slot for the forecast

How the four combine

Each factor is scored between 0 and 1. Each weight is a number the administrator chooses. The score is their sum.

$$\text{priority} = W_{\text{fair}} \times \text{fairshare} + W_{\text{age}} \times \text{age} + W_{\text{size}} \times \text{size} + W_{\text{class}} \times \text{class}$$

Suppose this machine is configured with these weights:

fairshare **100 000**

age **50 000**

job size **10 000**

service class **100 000**

Group A · used a lot, waiting two days

fairshare $0.10 \times 100\,000 = 10\,000$

age $0.29 \times 50\,000 = 14\,500$

size and class = 22 000

46 500

Group B · barely used, waiting two hours

fairshare $0.90 \times 100\,000 = 90\,000$

age $0.01 \times 50\,000 = 500$

size and class = 22 000

112 500

Same size, same service class. Recent usage is worth more here than two days of waiting.

The ratio between those weights is the allocation policy.



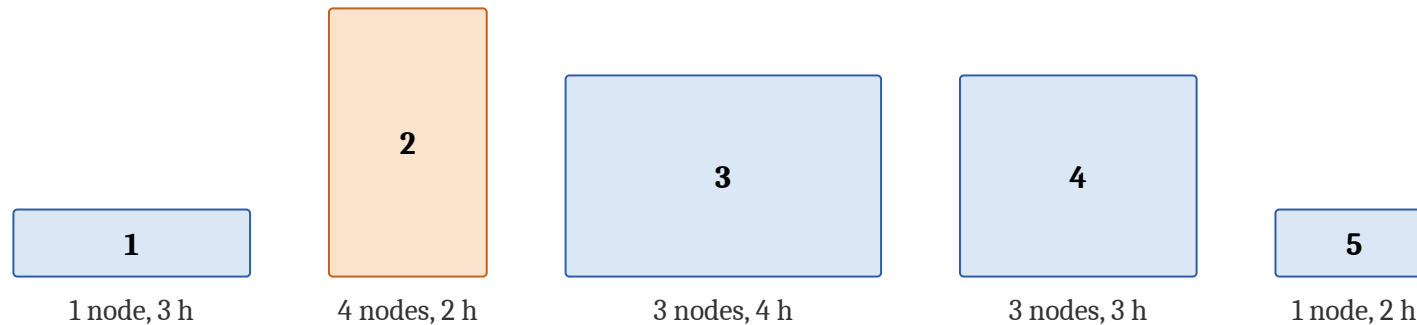
Section 4

Backfill

4

Four nodes, five requests

Every request is a rectangle: **nodes tall, hours wide**. Five arrive at once, already in priority order.



The area is the bill

nodes × hours

34 node hours in total

3 and 4 want the same three nodes. One asked for four hours, the other **for three.**

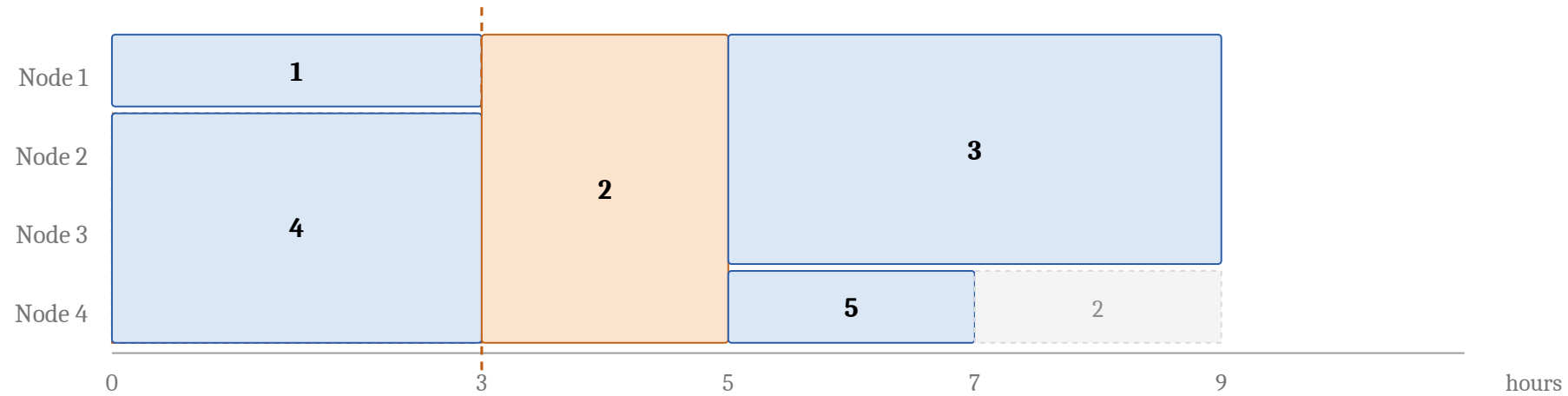
Strict priority: twelve hours



Request 2 needs all four nodes, so it waits — and everything behind it waits with it.

Twelve hours · 14 node hours idle · utilisation 70.8 %

Backfill: the same five, nine hours



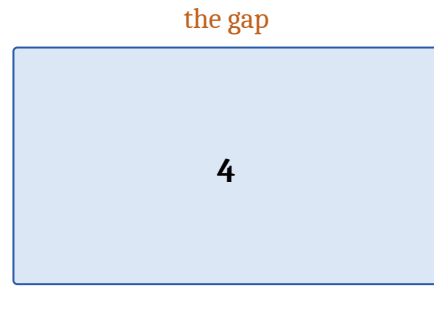
Request 2 is promised this moment, and the promise holds

Nine hours · 2 node hours idle · utilisation 94.4 %

Same work, same priorities, nobody disadvantaged — and about a third more machine.

A shorter request fits more gaps

The gap in front of the reservation was three nodes wide and three hours long.



asked for three hours

it fits, so it starts now



asked for four hours

it would run past the promise

Request 3 outranked request 4 the entire time, and started five hours later.

A shorter request buys no priority. It makes the work fit.

With one caution: ask for too little and the system stops the job at the limit you gave it.



Section 5

What this means

5

Three ways capacity gets destroyed

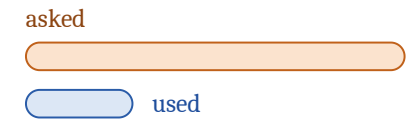
One GPU on a whole node

Four are billed, one is used. Three quarters of that spend produces nothing. A habits problem.



Time requests far above what the work needs

Free in money, expensive in waiting: it disqualifies the job from every gap.



Work run on the front desk

Four of them for the whole machine, and everyone's session shares them.

Hardware is charged whether you use it. Time is charged as consumed. **Be generous with time, precise about hardware.**

Four questions, always answerable

On any machine running SLURM, four things can be checked on demand.

What is free right now, and what does demand look like?

sinfo

Why is this project waiting, factor by factor?

sprio

Turns “we are being throttled” into a claim you can check.

When does the scheduler expect this to start?

squeue

What did this actually consume, against what it asked for?

sacct

The cheapest efficiency audit in the stack. Run it across a quarter.

Thank you

Questions welcome

Carlos Cotrini · Presented by Ghali

